Prism on a Laptop:

Streaming Inference of a 105B FP8 Mixture-of-Experts Decision Model and a Study of Expert Pruning

Pranav Krishnakumar

November 2026

Abstract

Prism is a 105B-parameter mixture-of-experts decision model that returns a probability for every option of a typed question in one forward pass. Its FP8 weights occupy 106.2 GB and the published implementation assumes a datacentre GPU with about 120 GB of memory. We ask two questions: whether the model can run correctly on a laptop, and how much of it is needed. We build a streaming engine that reads one decoder layer at a time from an NVMe SSD and computes on an Intel Arc 140V integrated GPU. Because a decision needs no decode loop, each layer is read once per batch, and batches amortise the read cost. The full model reaches 32.02 tokens per second at batch size 64 and reproduces the argmax of the model card example, with a maximum absolute probability gap of 0.041383. Against the hosted API, the full model agrees on 0.9267 of 1,200 questions with mean KL 0.0474. We then remove routed experts by router saliency. Keeping 32 of 128 experts gives 0.7750 argmax agreement with the full model, but the loss is uneven: NLI, sentiment and intent routing stay above 0.90 agreement, while knowledge multiple choice falls to 0.4080 and Hindi, Bengali and Telugu fall to 0.6275, 0.6373 and 0.5833. The calibration domain matters more than the ranking rule: geometry-calibrated experts score 0.9683 on held-out geometry questions but 0.6508 on general questions. Classification-style decisions survive aggressive expert pruning; knowledge and multilingual decisions do not.

1 Introduction

A decision model answers a question whose options are fixed in advance. It returns one probability per option in a single forward pass and never generates text. This interface fits routing, triage, moderation and scoring, where the set of answers is known and a calibrated probability is more useful than a sentence [10]. Prism is such a model, built on the 105B-parameter Sarvam mixture-of-experts backbone. Its released weights are FP8, occupy 106.2 GB, and the model card asks for one GPU with about 120 GB of memory. A laptop has 16 GB of shared graphics memory and 32 GB of system RAM. The first question of this report is whether the model can run correctly on that laptop.

The obstacle is memory, not arithmetic. The model is larger than the machine, so the weights must be read from disk. Offloading systems for generative inference already do this, but they are designed around a decode loop: each generated token needs another pass through every weight, and the KV cache grows with the sequence [1, 12, 13]. A decision model removes that loop. One request produces all option probabilities at once, so each layer's weights are read once per batch and the read cost is shared by every question in the batch. This difference makes a simple layer-streaming design practical: keep one decoder layer on the device, prefetch the next from an NVMe SSD, and discard it after use. The disk floor becomes the lower bound on latency, and batch size becomes the control that amortises it.

Correctness is a separate problem. Prism does not use a standard causal mask: the context is bidirectional, options share positions and cannot attend to each other, and an answer marker attends to the context and all options. The attention is a multi-head latent representation, the MoE router selects 8 of 128 experts with sigmoid scores, and the weights are FP8 with per-row scales [8]. The

integrated GPU has no FP8 matrix multiply, so the engine dequantises weights just in time and emulates FP8 activation quantisation in BF16. We therefore compare the engine with an independent FP32 CPU reference layer by layer, and the full model with the hosted API on 1,200 test questions.

The second question is how much of the model is needed. Prism routes each token to 8 of 128 experts in every MoE layer, and the routed experts account for 99,988,013,056 of the 106,216,275,456 weight bytes. We rank experts by router statistics collected on a calibration set, rewrite the router and expert tensors, and measure the pruned model against the full model. We vary the number kept, the ranking rule, the calibration domain, and the number of decoder layers. The measurements separate tasks that survive pruning from tasks that do not, and they show that the calibration distribution matters more than the ranking rule.

The report makes three contributions. First, it describes a streaming engine that runs the full 106.2 GB model on a laptop integrated GPU, with layer prefetch, lookup-table dequantisation, the Prism attention mask, and a CPU decision head. Second, it verifies the engine against an FP32 reference and the hosted API, including an honest account of a 0.041383 gap to the model card example. Third, it reports a controlled expert-pruning study over 1,200 test questions in nine domains and five languages. The main measurement is that keeping 32 of 128 experts preserves classification-style decisions (NLI 0.9800, sentiment 0.9875, intent routing 0.9036) but not knowledge multiple choice (0.4080) or Hindi, Bengali and Telugu (0.6275, 0.6373 and 0.5833). At 16 experts, overall agreement falls to 0.6633 and the accuracy drop grows from 11.00 to 18.83 percentage points.

2 Background

Prism is a decision model built on the Sarvam 105B mixture-of-experts backbone [10]. A request contains a state (text or compact JSON), one or more typed questions, and the options for each question. The encoder writes the state and question once, then lays out each option as its own span. Context tokens carry a segment embedding for the state or question, option tokens carry an option segment embedding, and the answer marker has its own segment. The decision head reads the hidden state at the closing token of each option and at the answer marker. This layout is the model's interface, so the engine transcribes the encoder rather than approximating it.

The attention mask follows the same layout. Context tokens attend to each other bidirectionally; each option attends to the context and its own tokens but not to other options; the answer marker attends to the context and every option. Options start at the same position and are mutually invisible, so they form a tree with a shared trunk. The mask is applied inside the attention softmax, and the answer token is the only place where option representations meet. Figure 1 shows the layout.

The backbone uses multi-head latent attention: queries and keys are split into a non-positional part and a rotary part, keys and values are compressed through a low-rank latent, and YaRN rotary tables extend the position range. Each of the 32 decoder layers has an attention block and an MLP block. Layer 0 is dense; layers 1 to 31 are sparse MoE layers with 128 routed experts and one shared expert. The router computes sigmoid scores, adds a correction bias, selects the top 8 experts, and normalises the selected weights to a fixed routed scale. The model stores routed experts in FP8 E4M3 with a per-row FP32 scale; embeddings and normalisation weights are BF16, and the router is FP32. The engine reproduces this mixed precision rather than casting the model to a single dtype.

The decision head receives the hidden state of each option's closing token and the answer marker, adds a learned question-type embedding, and applies two single-head self-attention blocks without positional encoding. The score of an option is the dot product between its output slot and the answer slot, scaled by the square root of the head width, followed by a softmax over options. Because the head has no positional encoding and the options share positions and cannot see each other, permuting the options permutes the output probabilities without changing them. This property is a useful test of the whole engine: if the encoder, mask or head is wrong, option-order invariance usually breaks.

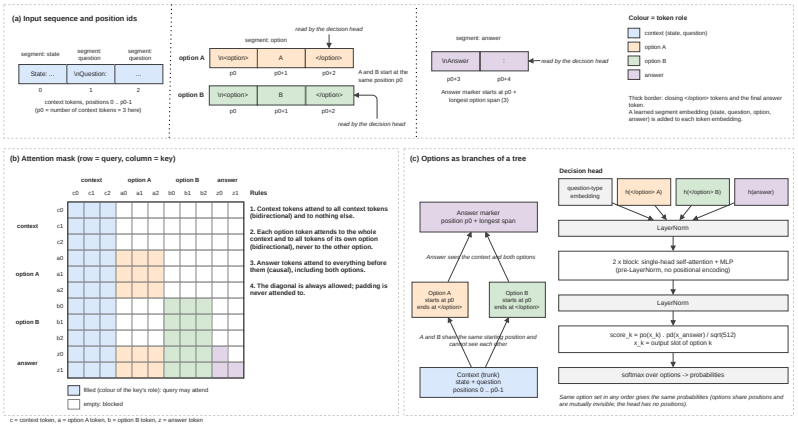


Figure 1: Input layout and attention mask for one question with two options. Context tokens attend bidirectionally; each option attends to the context and to its own tokens but not to the other option; the answer marker attends to the context and both options. Options share a start position, so they form branches of a tree. The decision head reads the closing token of each option and the answer marker.

3 Streaming Inference Engine

The engine reads the weights one decoder layer at a time. The backbone is split across 33 safetensors files: one embedding file and 32 decoder-layer files. The decision head is a separate file. The embedding file is 2.15 GB, layer 0 is 0.30 GB, and each MoE layer is 3.35 GB. A layer is staged in a pinned host buffer by 8 reader threads with 64 MB reads, copied to the integrated GPU, and released after its activations are computed. A background reader fills the next buffer while the current layer computes, so the disk read of layer $i + 1$ overlaps the arithmetic of layer i. The device holds one layer's weights and the current activations, never the full model.

This schedule is practical because a decision model has no decode loop. A generative model visits every layer once per generated token, so streaming would pay the read cost repeatedly and the KV cache would grow with the output. Prism produces all option probabilities in one forward pass. Every layer's weights are read exactly once per batch, and the same read serves every question in the batch. On the measured laptop the SSD delivers 1.2148 GB/s with 8 threads and 64 MB blocks, so one full pass of 104,068,783,616 layer bytes has a floor of 77.2 s. The three-question example takes 210.8 s in FP8-emulation mode, and most of the additional time is computation rather than I/O.

FP8 weights are dequantised with a lookup table. A 256-entry table maps each E4M3 byte to its exact value; 254 codes are finite, the largest finite value is 448.0 and the smallest positive value is 0.001953125. The alternative is a native cast to BF16, which the integrated GPU supports but which is slower in the measured reader loop: the lookup table reaches 0.8458 GB/s, the native cast 0.5063 GB/s and a bit-manipulation path 0.1041 GB/s. The table is exact for finite codes and produces the same values as the native cast. Routed experts are dequantised one expert at a time, after the router has selected the experts for the tokens in the batch, which keeps the peak device allocation small.

Attention follows the Prism mask. The engine keeps the RoPE tables on the device, projects queries, keys and values through the latent representation, applies rotary embeddings to the positional part, and computes the attention scores in chunks over the batch. Chunking bounds the attention matrix, whose size grows with the square of the sequence length, and does not change the result. The mask is a boolean matrix built from token roles, and the softmax is taken over allowed keys only. The attention workspace is discarded after each layer.

The MoE block routes tokens, sorts them by expert, and runs one grouped matrix multiply per non-empty expert, followed by the shared expert. The routed outputs are accumulated in FP32 and cast to BF16 once, which avoids compounding rounding across the eight selected experts. The router itself runs in FP32. The same code path collects router statistics when the engine is used for calibration, so the pruning study uses the router decisions of the full model.

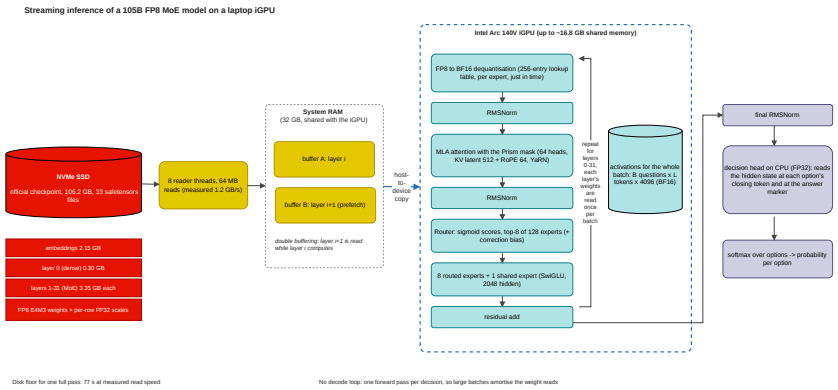


Figure 2: Data path of the streaming engine. The 106.2 GB weight set is read from the NVMe SSD by 8 threads in 64 MB blocks, with a measured read speed of 1.2148 GB/s. One decoder layer is resident on the integrated GPU; the next layer is prefetched into a second host buffer while the current layer computes. Routed experts are dequantised from FP8 to BF16 just in time, and the final norm and decision head run on the CPU in FP32.

The decision head runs on the CPU in FP32. It reads the final hidden state at each option's closing token and at the answer marker, applies the two attention blocks, and returns a softmax over options. Keeping the head on the CPU avoids a separate device kernel for a small computation and guarantees that the probabilities are produced in the same precision as the reference. The final RMSNorm is also applied on the CPU.

Two hardware details shaped the implementation. The integrated GPU shares system RAM with the CPU and has no FP8 matrix multiply, so weights are dequantised to BF16 before use and activations are quantised to the E4M3 grid only in emulation mode. After an out-of-memory error the Intel driver did not return the device to a usable state, and later allocations failed even after the tensors were freed. All GPU runs therefore use a fresh process, and the engine checks free memory before allocating a layer. Figure 2 shows the data path.

4 Verification

4.1 Layer-level reference

The reference is an independent FP32 CPU implementation that reads the same weights and computes each decoder layer with explicit loops over experts. It does not share code with the engine: the reference dequantises weights with a float32 multiply, applies the mask with a double loop over token roles, and accumulates the routed experts with index addition. We run the team question of the model card example (78 tokens) through both implementations and compare the hidden state after layers 0 and 1. Table 1 reports the minimum cosine and the relative L2 error over the sequence. The FP32 path matches the reference to 3.40×10^{-7} and 3.92×10^{-7} relative L2; BF16 to 2.92×10^{-3} and 3.78×10^{-3}; BF16 with FP8 activation emulation to 3.40×10^{-3} and 4.64×10^{-3}. Extending the check to layers 2 and 3 gives 4.79×10^{-3} and 5.51×10^{-3} in BF16, and 6.35×10^{-3} and 8.51×10^{-3} in FP8 emulation. At layer 3 the BF16 router-weight trace crosses the stored threshold on some tokens (minimum cosine 0.778, relative L2 0.0905), while the hidden state remains close; we treat this as a precision limit of BF16 routing, not a functional failure. The decision head is also checked against an independent implementation of the published head on random inputs: the maximum absolute difference is 4.48×10^{-9} for the choice head and 0 for the yes/no and score heads.

4.2 The model card example

Table 2 compares the engine with the three-question example published on the model card. The engine selects the same options: billing, yes and anger level 2. The maximum absolute difference is

Table 1: Hidden-state comparison against the FP32 CPU reference after layers 0 and 1. The sequence is the 78-token team question. Cosine is the minimum over tokens; relative L2 is over the whole hidden state.

Mode	L0 cos	L0 rel. L2	L1 cos	L1 rel. L2
FP32	0.9999998	3.40×10^{-7}	0.9999996	3.92×10^{-7}
BF16	0.9999920	2.92×10^{-3}	0.9999868	3.78×10^{-3}
BF16 + FP8 emulation	0.9999858	3.40×10^{-3}	0.9999689	4.64×10^{-3}

Table 2: The model card example in FP8-emulation mode. The largest absolute difference is 0.041383. The argmax choices match.

Field	Engine	Published
team: billing	0.986213	0.986
team: tech	0.007307	0.007
team: delivery	0.006481	0.006
urgent: yes	0.467766	0.45
anger: 0	0.035849	0.032
anger: 1	0.293908	0.267
anger: 2	0.520626	0.51
anger: 3	0.149617	0.191

0.041383, at anger level 3, where the engine gives 0.149617 and the model card gives 0.191. The other differences are smaller: urgent 0.467766 against 0.45, anger level 2 0.520626 against 0.51, and team billing 0.986213 against 0.986.

The gap has a history. An earlier revision measured 0.022238 on the same example; the current engine measures 0.041383. The preserved code was edited after that measurement, so it is numerically equivalent to the current engine and reproduces 0.041383. We therefore reverted the four edits one at a time in a scratch copy. Reverting the FP32 accumulation in the MoE to BF16 moves the FP8-emulation result from 0.041383 to 0.111615, the largest change of any revert. The activation-scale path changes the result by 9.9×10^{-6}, the RoPE layout is numerically neutral, and reverting the attention scale to the value without the YaRN multiplier gives 0.070594 in FP8 emulation and 0.050096 in BF16. On layers 0-3 the current engine is slightly closer to the FP32 reference than the preserved engine at layer 3 (relative L2 0.005507 against 0.006800 in BF16, and 0.008513 against 0.009421 in FP8 emulation), with mixed differences at the earlier layers. The larger published-example gap is a change in accumulation order, not a loss of layer-level accuracy. The exact 0.022238 combination is not preserved, so we report it as history.

The gap is smaller than the hosted API's own gap to the model card. The API returns 0.982509 for team billing, 0.369260 for urgent yes, and 0.030670, 0.292758, 0.513575, 0.162997 for the four anger levels; its maximum absolute difference from the published example is 0.080740. A 0.041383 engine gap is therefore not evidence that the engine is wrong, and the layer-level comparison is the stronger correctness check. It does mean that the model card probabilities should be treated as a reference point, not as an exact target.

4.3 Invariance and API parity

The earlier revision passed four invariance checks: option order (maximum absolute difference 0.000761), determinism (0.0), batch composition (0.000506) and padding (0.0000435). The current engine uses the same encoder, mask and head, so the mechanism is unchanged, but we did not repeat the full sweep on the current revision. The option-order property follows from the design: options share positions, cannot attend to each other, and the head has no positional encoding.

Table 3 compares the engine with the hosted API on the 1,200 questions that are present in every run. The full model agrees with the API on 0.9267 of questions, with mean KL 0.0474 and mean total

Table 3: Parity with the hosted Sarvam Prism API on 1,200 questions. The reference is the API. KL is KL(API ‖ system); the delta is system accuracy minus API accuracy in percentage points.

System	Agreement	Mean KL	Mean TV	API acc.	Delta (pp)
Full	0.9267	0.0474	0.0654	0.8175	+0.17
$N = 32$ general	0.7775	0.2773	0.1907	0.8175	−10.83
$N = 16$ general	0.6608	0.4937	0.2877	0.8175	−18.67
$N = 32$ geometry	0.6508	0.5388	0.3027	0.8175	−20.17

Table 4: Throughput on the tested laptop. The full model uses the disk-streaming mode on the integrated GPU; the pruned variants use the same mode. The CPU row uses RAM-resident weights. Effective TFLOPS is the reported engine metric.

Configuration	Device	Batch	Tokens	Total (s)	Tokens/s
Full model	iGPU, disk	3	402	134.8	2.98
Full model	iGPU, disk	16	2,162	146.3	14.78
Full model	iGPU, disk	64	8,662	270.5	32.02
$N = 32$	iGPU, disk	1	136	33.5	4.06
$N = 32$	iGPU, disk	16	2,230	45.2	49.33
$N = 32$	iGPU, disk	64	8,918	98.6	90.48
$N = 16$	iGPU, disk	1	136	21.3	6.39
$N = 16$	iGPU, disk	16	2,230	26.2	85.09
$N = 16$	iGPU, disk	64	8,918	79.7	111.89
$N = 16$	CPU, RAM	3	419	63.2	6.63

variation 0.0654. The API accuracy is 0.8175 and the full engine accuracy is 0.8192, a difference of 0.17 percentage points. The pruned variants fall below the full model on the same comparison: $N = 32$ general agrees on 0.7775, $N = 16$ general on 0.6608, and $N = 32$ geometry-calibrated on 0.6508. The 50-repeat noise floor is an argmax agreement of 1.0000, mean absolute difference 0.001531, mean KL 0.000324 and mean total variation 0.003252, so the full-model gap is well above API noise.

5 Performance

All measurements use the same laptop: an Intel Core Ultra CPU, 32 GB of system RAM, an Intel Arc 140V integrated GPU with 16 GB of shared memory, and an NVMe SSD that reads at 1.2148 GB/s with 8 threads and 64 MB blocks. The engine uses BF16 weights after dequantisation and a lookup-table dequantiser. The full model reads 104,068,783,616 bytes per pass, so its disk floor is 77.2 s. Table 4 reports throughput and Figure 3 plots it against batch size.

Batch size is the dominant control. On the full model, throughput rises from 2.98 tokens per second at batch 3 to 32.02 at batch 64. The pruned models scale further because their weights are smaller: $N = 16$ goes from 6.39 tokens per second at batch 1 to 111.89 at batch 64. The full model is compute-bound at the largest batch: its effective throughput is 0.920 TFLOPS against a measured BF16 matmul peak of 12.860 TFLOPS. The $N = 32$ and $N = 16$ variants reach 2.133 and 2.553 effective TFLOPS. The disk floor is 77.2 s of the 270.5 s total, so further disk tuning alone cannot make the full model much faster.

Figure 4 separates the disk floor from the rest of a full pass at batch 64. The floor is 77.2 s and the total is 270.5 s, leaving 193.3 s for layer computation, host-to-device copies and other overhead. The same split at batch 3 is a 77.2 s floor against a 134.8 s total, and at batch 16 a 146.3 s total. The extra time grows with the number of tokens, which is why larger batches improve tokens per second while increasing latency per batch. The three-question example takes 210.8 s in FP8-emulation mode and 151.0 s in BF16.

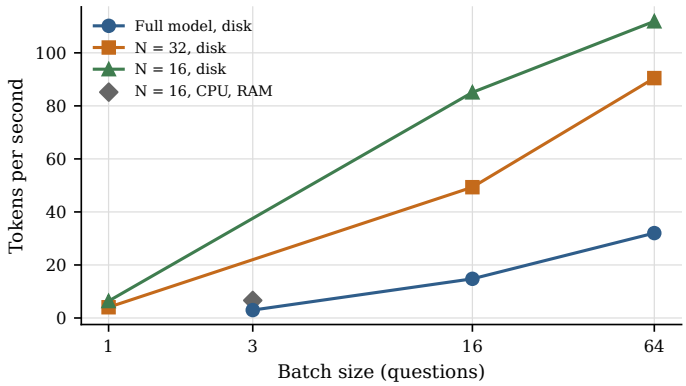


Figure 3: Tokens per second against batch size for the full model and the two pruned variants on the integrated GPU, plus the CPU-only $N = 16$ point. The horizontal axis is logarithmic in batch size.

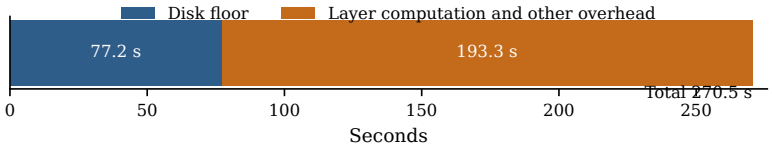


Figure 4: Time of one full-model pass at batch size 64 against the measured disk floor. The orange segment includes layer computation, host-to-device copies and other overhead.

The CPU-only result is a fallback rather than a throughput competitor. $N = 16$ on the CPU with RAM-resident weights reaches 6.63 tokens per second at batch 3 and peaks at 18.54 GB of resident memory; the integrated GPU reaches 85.09 tokens per second on the same variant at batch 16. The CPU path does not need the SSD after loading, so it remains useful when the GPU is unavailable.

6 Expert Pruning

6.1 Method

The routed experts dominate the weight set: the expert tensors account for 99,988,013,056 of 106,216,275,456 bytes. We keep the first N experts of every MoE layer under a ranking computed from a calibration pass, and rewrite the router and expert tensors so that the pruned model is a valid weight set of the same architecture with 128 replaced by N. Layer 0 is dense and is copied unchanged. The kept sets are nested: the $N = 16$ set is a subset of $N = 32$, and $N = 8$ is a subset of $N = 16$, so differences between sizes are not confounded by a different ranking.

The ranking uses two statistics per expert and layer. During a calibration pass we accumulate the router weight $g_j(t)$ and the norm $\|f_j(t)\|$ of the expert output for every token routed to expert j. The saliency sum is $S_j = \sum_t g_j(t)\|f_j(t)\|$. The REAP rule ranks by S_j/c_j, the mean saliency per activation, where c_j is the number of tokens routed to the expert [6]. The TOTAL rule ranks by S_j/T, the saliency divided by the number of tokens in the layer. TOTAL favours experts that are used often and strongly; REAP favours experts that are strong when they are used.

We use two calibration sets. The general set is 500 questions drawn from the same eleven public datasets as the test set, interleaved across datasets and languages; it contains 82,555 tokens. The narrow set is 96 synthetic geometry questions with 18,760 tokens. The geometry questions are a stress test for a decision task far from the general distribution, not a claim that geometry is the target application. The geometry statistics have a mean zero-count fraction of 0.0449 and a maximum of 0.1875, so some experts are never selected on that set; the general set has a mean of 0.000756 and a maximum of 0.015625. The mean kept-set Jaccard overlap between the two calibrations is 0.3212 at $N = 32$ and 0.2638 at $N = 16$, so they should be treated as different experts, not two views of the

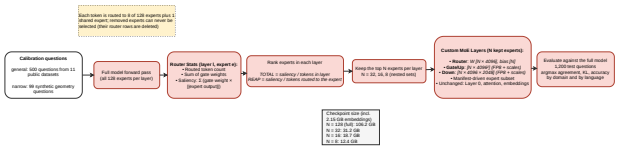


Figure 5: Expert pruning pipeline. A calibration pass collects router and expert-output statistics for every MoE layer. Experts are ranked by TOTAL or REAP saliency, the top N are kept in each layer, and the router and expert tensors are rewritten. The pruned weight set is compared with the full model on the test questions.

Table 5: Fidelity of pruned variants against the full model on 1,200 test questions. Agreement is the fraction of identical argmax choices; KL is KL(full ‖ variant); TV is total variation. The 95% interval is for agreement. Accuracy is on the labelled test set.

Variant	N	Agreement	95% CI	Mean KL	Mean TV	Acc. full	Acc. var.
N32_general	32	0.7750	0.7505–0.7977	0.2866	0.1949	0.8192	0.7092
N16_general	16	0.6633	0.6361–0.6895	0.5148	0.2963	0.8192	0.6308
N32_total	32	0.6508	0.6234–0.6773	0.5626	0.3112	0.8192	0.6158
N16_total	16	0.5742	0.5460–0.6019	0.7535	0.4091	0.8192	0.5500
N16_reap	16	0.6342	0.6065–0.6610	0.5960	0.3494	0.8192	0.5925
N8_total	8	0.4925	0.4643–0.5208	0.8967	0.4823	0.8192	0.4733

same set.

The pruned weight sets are verified in three ways. The router and expert tensors have the expected shapes and the kept ids are sorted, unique and in range. The file sizes match the size arithmetic to a relative error of at most 8.43×10^{-6} (29,029,090,688 bytes for $N = 32$, 16,522,459,864 for $N = 16$, 10,269,144,576 for $N = 8$). An identity check rewrites layers 0 and 1 with $N = 128$ and $N = 16$ and confirms that the hidden state after layer 1 is bitwise identical to the original for the three-question example.

6.2 Fidelity

The reference is the full BF16 engine with identical batching. We report argmax agreement, the fraction of questions on which the pruned model and the full model select the same option; mean KL divergence KL(full ‖ variant); mean total variation; and accuracy on the labelled test set. The test set has 1,200 questions in nine domains and five languages. The overall results are in Table 5 and Figure 6. Keeping 32 of 128 experts gives 0.7750 agreement and an accuracy drop of 11.00 percentage points. Keeping 16 gives 0.6633 agreement and a drop of 18.83 points. The REAP ranking at $N = 16$ is worse than TOTAL on this test set, with 0.6342 agreement and 0.5960 mean KL against 0.6633 and 0.5148 for TOTAL. The $N = 8$ variant reaches 0.4925 agreement and loses 34.58 points of accuracy.

The average hides the main result. Figure 8 splits the general-calibrated variants by domain. At $N = 32$, NLI reaches 0.9800 agreement, sentiment 0.9875, topic or entity type 0.9571, intent routing 0.9036, reading-comprehension QA 0.8875, tool-use decision 0.8286 and topic classification 0.8167. Knowledge multiple choice reaches only 0.4080, and toxicity or moderation 0.6933. The accuracy drop follows the same split: NLI loses 0.00 points, sentiment 1.25, topic or entity type 4.29, intent routing 4.64, reading QA 1.25, topic classification 0.83 and toxicity 3.33, while knowledge MCQ loses 40.00 and tool use 11.43. At $N = 16$ the classification-style tasks remain usable but the knowledge task collapses further, to 0.3080 agreement and a 46.00-point drop.

The language split is in Figure 9. English agrees at 0.8225 at $N = 32$ and 0.7193 at $N = 16$. Tamil is close to English at $N = 32$ (0.8065) but falls to 0.5323 at $N = 16$. Hindi, Bengali and Telugu are already below 0.65 at $N = 32$ (0.6275, 0.6373 and 0.5833) and fall to 0.5490, 0.5294 and 0.4583 at $N = 16$. The accuracy drops are larger than the English drop at both sizes: 20.59, 16.67 and 25.00 points at $N = 32$ for Hindi, Bengali and Telugu, against 8.35 for English. The calibration set is 71% English; the low-resource languages lose experts that the general calibration rarely selects.

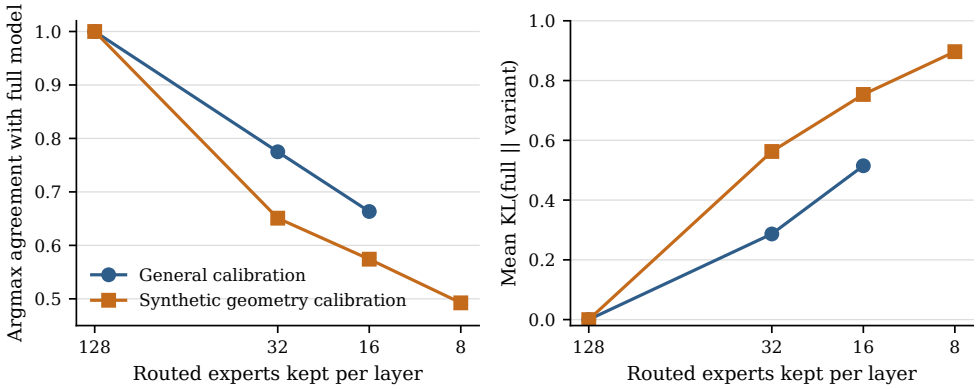


Figure 6: Agreement and KL against the full model as routed experts are removed. The general-calibration curve has no $N = 8$ point because that variant was not built; the synthetic-geometry curve includes $N = 8$. The full model is the identity reference at $N = 128$.

6.3 Calibration and ranking

Cross-evaluating the two calibration domains gives the following. General calibration is the better choice on the general test set at both sizes: 0.7750 against 0.6508 at $N = 32$, and 0.6633 against 0.5742 at $N = 16$. The order reverses on the 63 held-out geometry questions: geometry calibration gives 0.9683 agreement at $N = 32$, while general calibration gives 0.6984 at $N = 32$ and 0.4762 at $N = 16$. The general-calibrated $N = 32$ model also has a much larger geometry KL (0.5779) than the geometry-calibrated model (0.0261). The calibration-domain contrast is larger than the TOTAL-versus-REAP contrast at either size. The calibration distribution, not the ranking rule, is the first decision when pruning this model.

6.4 Layer skipping

A second way to shrink the model is to skip whole decoder layers, which removes attention and MoE cost rather than experts. On the 63 held-out geometry questions, the unskipped $N = 32$ geometry-calibrated model reaches 0.9683 agreement, 0.0261 KL and 0.7302 accuracy at 70.1 tokens per second. Skipping layers 20 and 28 reduces agreement to 0.7143 and raises KL to 0.4141; skipping layers 20, 25, 28 and 29 gives 0.7302 agreement and 0.4656 KL. The four-layer skip is not worse than the two-layer skip on this small set, so the ordering is not monotonic in the number of layers. What is clear is that layer skipping costs much more fidelity than expert pruning at comparable size: skipping two layers drops agreement by 0.253968, while the 32-expert model agrees at 0.9683 on the same 63 questions. We report the measurement and do not claim a general rule from 63 questions.

7 Related Work

Expert pruning and merging. Sparse MoE layers trade parameters for compute: each token activates a small subset of experts [4, 5, 11]. The static parameter count still dominates memory, so a line of work removes experts after training. Lu et al. prune experts by reconstruction loss on a calibration set and combine pruning with dynamic expert skipping [7]. REAP ranks experts by the router-weighted norm of their output and argues that pruning preserves routing control that merging loses [6]. MoE-Pruner scores individual weights with router weights and input activations [15]. Our study uses the same router statistics but asks which decision tasks survive when entire experts are removed, and how much the calibration distribution changes the answer.

Weight offloading and layer streaming. DeepSpeed-Inference combines GPU, CPU and NVMe memory to serve models larger than total GPU memory [1]. FlexGen targets high-throughput

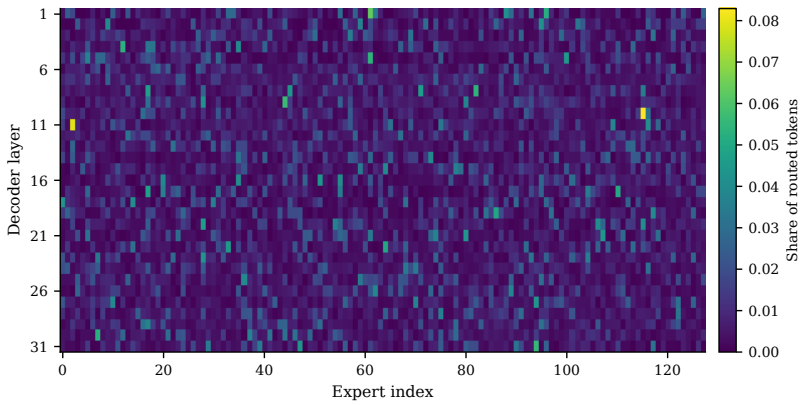


Figure 7: Share of routed tokens per expert and layer on the 500-question general calibration set. Each row is a MoE layer and each column an expert; the share is the expert's routed-token count divided by all routed tokens in that layer. Usage is spread across the 128 experts rather than concentrated on a fixed small set.

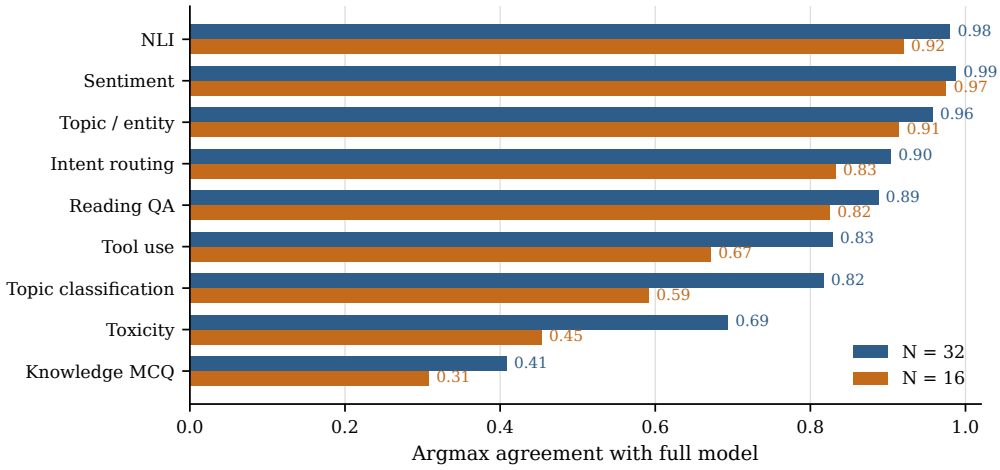


Figure 8: Argmax agreement by domain at $N = 32$ and $N = 16$ for the general-calibrated variants. The tasks are ordered by $N = 32$ agreement.

generation on a single commodity GPU and searches over tensor placement and batch schedules [12]. PowerInfer exploits activation locality to keep hot neurons on the GPU and compute cold neurons on the CPU [13]. For MoE models, Eliseev and Mazur exploit the sparse activation pattern to cache and prefetch experts during generation [3]. These systems are built around autoregressive decoding, where the weights are revisited for every token and the KV cache is a first-class object. A decision model has no decode loop: one forward pass returns every option probability, so streaming can be organised as a single sequential read of the weights. Our engine is deliberately simple for this setting, and its measurements expose the disk floor and the batch amortisation directly.

Classification heads and decision models. Encoder models such as BERT are usually adapted to classification with a small head on a pooled representation [2]. Sentence-BERT and SetFit show that strong sentence representations plus a light classifier can replace heavier fine-tuning for many classification tasks [9, 14]. Prism differs in that it scores the options jointly: the option spans share a context, cannot see each other, and meet only at the answer marker, and the head attends over the option slots. This design gives a calibrated distribution over a fixed option set in one pass [10]. Our pruning results are therefore not a generic statement about classification heads; they measure how much of a 105B joint scorer is needed for decisions of different kinds.

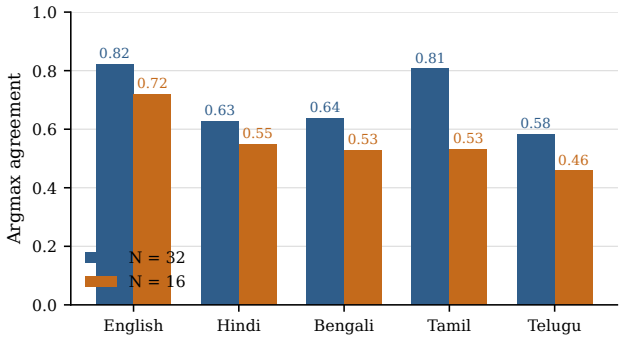


Figure 9: Argmax agreement by language at $N = 32$ and $N = 16$ for the general-calibrated variants.

8 Limitations

The measurements come from one laptop with one integrated GPU and one SSD. We did not test NVIDIA hardware, a discrete GPU, or a different disk. The FP8 path is emulated in BF16 because the integrated GPU has no FP8 matrix multiply; native FP8 hardware would change both speed and numerics. The engine is not fast in absolute terms: a full pass at batch 64 takes 270.5 s, a three-question request takes 210.8 s in FP8-emulation mode, and the disk floor alone is 77.2 s. The pruned variants are faster but are not released.

The test set is bounded: 1,200 questions in nine domains and five languages, with a longest question of 569 tokens. The calibration sets are 500 general questions and 96 synthetic geometry questions. The 63 held-out geometry questions are a small evaluation set, and the layer-skip comparison is a measurement on that set, not a general rule. We did not test other Indian languages, long contexts, code, security prompts, or top-k values other than 8. The host-API parity uses recorded responses and a fixed test set; the hosted service may change. The invariance sweep was run on the earlier revision, although the encoder, mask and head are unchanged.

The golden example is not an exact reproduction. The current engine is 0.041383 away from the published probabilities, and the hosted API is 0.080740 away. The layer-level reference and API parity are stronger evidence than the model card example, but the gap means that the engine matches the model's decisions rather than every published probability. The pruning results are also specific to this model and router: TOTAL and REAP are first-order saliency heuristics, and the low overlap between general and geometry calibration shows that the ranking is sensitive to the calibration distribution. A production system would need a calibration set that covers its languages and decision types, with a held-out check for each.

9 Conclusion

This report asked whether a 105B FP8 decision model can run on a laptop and how much of it is needed. The streaming engine answers the first question: one decoder layer at a time, with prefetch and just-in-time FP8 dequantisation, runs the full 106.2 GB model on an integrated GPU at 32.02 tokens per second at batch 64, matches the model card's argmax choices, and agrees with the hosted API on 0.9267 of 1,200 questions. The layer-level reference and the API comparison show that the engine implements the model, not an approximation of it. The remaining 0.041383 gap to the model card is smaller than the hosted API's own 0.080740 gap, and the layer-level comparison shows that the current engine is not less accurate than the preserved revision at the deepest layer checked.

The pruning study answers the second question with a boundary rather than a single number. Keeping 32 of 128 experts preserves classification-style decisions: NLI at 0.9800 agreement, sentiment at 0.9875, intent routing at 0.9036, and topic or entity type at 0.9571. It does not preserve knowledge multiple choice, which falls to 0.4080 and loses 40.00 accuracy points, or Hindi, Bengali and Telugu, which fall to 0.6275, 0.6373 and 0.5833. The calibration domain is the first variable: geometry-calibrated experts

Table 6: Weight-set size for different expert counts. The total column is the 32-layer backbone; the embedding column adds the 2,147,483,648-byte embedding file.

N	Layer bytes	31 layers	Total bytes	With embedding (GB)
128	3,347,500,800	103,772,524,800	104,068,783,616	106.216
32	926,862,720	28,732,744,320	29,029,003,136	31.176
16	523,423,040	16,226,114,240	16,522,373,056	18.670
8	321,703,200	9,972,799,200	10,269,058,016	12.417

reach 0.9683 on held-out geometry but 0.6508 on the general test, and general-calibrated experts show the reverse. The ranking rule is second: TOTAL beats REAP at $N = 16$ on the general test and at $N = 32$ on geometry, but by a smaller margin.

The practical implication is that a pruned decision model should be calibrated for the decisions and languages it will serve, and that a single "experts kept" number is not a sufficient specification. A useful next step is a mixed calibration set with per-language and per-domain held-out checks, and a native FP8 implementation on hardware that supports it. The engine and the result files are public, so both the streaming path and the pruning measurements can be reproduced without the original machine.

A Tensor Inventory and Reproduction

A.1 Inventory

The weight set contains 732 backbone tensors in 33 files: one embedding file and 32 decoder-layer files; the decision head is a separate file. The backbone totals 106,216,275,456 bytes; the decision head adds 117,837,824 bytes. The tensor categories are 2,147,483,648 bytes of embedding, 3,032,588,288 of attention, 99,988,013,056 of routed experts, 781,156,352 of shared experts, 65,027,584 of router, 201,474,048 of dense MLP, 524,288 of normalisation and 8,192 of final norm.

A.2 Size arithmetic

Let $b_e = 25{,}198{,}592$ be the bytes of one routed expert, $b_r = 16{,}388$ the bytes of one router row, $b_s = 25{,}198{,}592$ the bytes of the shared expert and $b_a = 94{,}784{,}768$ the bytes of the attention block. One MoE layer with N kept experts has

$$B_{\text{layer}}(N) = N(b_e + b_r) + b_s + b_a. \tag{1}$$

The 32-layer backbone is

$$B_{\text{total}}(N) = 31\, B_{\text{layer}}(N) + B_{\text{layer }0}, \tag{2}$$

with $B_{\text{layer }0} = 296{,}258{,}816$ bytes. Table 6 evaluates this for the sizes used in the study. The reduction is relative to $N = 128$.

A.3 Environment and reproduction

The measurements use Python 3.11, torch 2.9.1+xpu, driver 1.6.34541, an Intel Core Ultra CPU, 32 GB of system RAM, an Intel Arc 140V integrated GPU with 16 GB of shared memory, and an NVMe SSD. The measured BF16 matmul throughput is 12.860 TFLOPS at 4096^3, the host-to-device copy rate is 0.7924 GB/s, and the disk read rate is 1.2148 GB/s with 8 threads and 64 MB blocks. The engine has not been tested on NVIDIA hardware.

The public repository contains the engine, the result files and the scripts that produced them. The main commands, run from the repository root, are:

```
python scripts/inventory.py --weights weights
python scripts/verify_layers.py --weights weights
python scripts/golden.py --weights weights
python scripts/perf.py --weights weights --out results
python scripts/router_stats.py --weights weights \
  --data data/eval/calib.jsonl --out results
python scripts/build_pruned.py --weights weights \
  --stats results/pruning/router_stats.npz --ranking total \
  --n 32,16 --suffix general --out results
python scripts/run_eval.py --weights weights \
  --variant N32_general --split test --out results
python scripts/eval_fidelity.py --probabilities results/probabilities \
  --labels results/eval/labels.json --out results
python scripts/api_parity.py
```

The complete command list is in the repository's README. The evaluation set is built from public datasets with their licences recorded in `results/eval/datasets.json`; question text is not redistributed.

References

[1] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC)*, pages 1–15, 2022.

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 4171–4186, 2019.

[3] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. arXiv preprint arXiv:2312.17238, 2023.

[4] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[5] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian El Sayed, et al. Mixtral of experts. arXiv preprint arXiv:2401.04088, 2024.

[6] Mike Lasby, Ivan Lazarevich, Nish Sinnadurai, Sean Lie, Yani Ioannou, and Vithursan Thangarasa. REAP the experts: Why pruning prevails for one-shot MoE compression. In *International Conference on Learning Representations*, 2026.

[7] Xudong Lu, Qi Liu, Yuhui Xu, Aojun Zhou, Siyuan Huang, Bo Zhang, Junchi Yan, and Hongsheng Li. Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 6159–6172, 2024.

[8] Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, et al. FP8 formats for deep learning. arXiv preprint arXiv:2209.05433, 2022.

[9] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992, 2019.

[10] Sarvam AI. Sarvam-Prism. Model card, Hugging Face, 2026. URL `https://huggingface.co/sarvamai/prism`. Apache-2.0.

[11] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

[12] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *International Conference on Machine Learning*, pages 31094–31116, 2023.

[13] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. PowerInfer: Fast large language model serving with a consumer-grade GPU. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 590–606, 2024.

[14] Lewis Tunstall, Nils Reimers, Unso Eun Seo Jo, Luke Bates, Daniel Korat, Moshe Wasserblat, and Oren Pereg. Efficient few-shot learning without prompts. arXiv preprint arXiv:2209.11055, 2022.

[15] Yanyue Xie, Zhi Zhang, Ding Zhou, Cong Xie, Ziang Song, Xin Liu, Yanzhi Wang, Xue Lin, and An Xu. MoE-Pruner: Pruning mixture-of-experts large language model using the hints from its router. arXiv preprint arXiv:2410.12013, 2024.